UNIVERSIDAD DE SAN CARLOS DE
GUATEMALA FACULTAD DE INGENIERÍA
PRACTICAS INICIALES F-

NOMBRE:

MARIO JACOBO GONZÁLEZ IXCAYAU - 202307811
JULIO FERNANDO EFRAÍN CHAMORRO CAJCHUM -201632179
SOFIA GUTIERREZ ROGEL - 202404463
KEVIN EMANUEL RAMIREZ REVOLORIO - 201907719
OLIVERIO MACZ COC - 202002019
LUIS ARMANDO GAITÁN LAZÁRO - 202404079

TUTORES:

THANYA MAZARIEGOS, MARÍA FERNANDA MORALES, BRANDON MARROQUÍN

GUATEMALA, 26 DE MARZO DEL 2026

Manual Técnico — Proyecto Web

Resumen ejecutivo

Aplicación web de tipo muro social académico desarrollada para estudiantes de la Facultad de Ingeniería de la Universidad de San Carlos de Guatemala. La aplicación permite a los usuarios publicar opiniones sobre cursos y catedráticos, así como interactuar mediante comentarios y consultar perfiles académicos.

El frontend está construido con React 18 y React Router 6; el backend con Node.js (Express) y persistencia en MySQL. Se implementa autenticación mediante JWT, almacenamiento seguro de contraseñas con bcrypt y gestión de cursos aprobados con cálculo de créditos acumulados.

Objetivo del sistema

El objetivo del sistema es proporcionar una plataforma donde estudiantes puedan consultar y compartir información sobre cursos y catedráticos, centralizando opiniones que normalmente se encuentran dispersas en redes sociales.

1. Requerimientos técnicos

Los requerimientos técnicos para ejecutar el sistema son los siguientes:

- **Sistema operativo:** Windows 10 o superior, Linux o macOS.
- **Node.js:** Versión 14 o superior instalada en el sistema.
- **Gestor de paquetes (npm):** Incluido con Node.js, necesario para instalar dependencias del proyecto.
- **MySQL Server:** Sistema de gestión de base de datos para el almacenamiento de la información.
- **MySQL Workbench (opcional):** Herramienta gráfica para la administración de la base de datos.
- **Navegador web:** Google Chrome, Microsoft Edge o cualquier navegador moderno para visualizar la aplicación.
- **Editor de código (opcional):** Visual Studio Code u otro editor compatible para visualizar o modificar el código fuente.
- **Acceso a terminal o línea de comandos:** Necesario para ejecutar el backend y frontend del sistema.
- **Conexión a internet:** Requerida para instalar dependencias del proyecto.

2. Arquitectura general

- **Frontend:** Create React App (CRA), React 18, React Router 6, `jwt-decode` para leer el JWT en el navegador. Estilos con CSS (archivos `home.css`, `perfil.css`, `publi.css`).

- **Backend:** Express + middlewares (`cors`, `body-parser`), `mysql2` para conexión a base de datos, `bcrypt` para hashing y `jsonwebtoken` para JWT.

- **Base de datos:** MySQL con tablas `usuarios`, `cursos`, `catedraticos`, `publicaciones`, `comentarios`, `cursos_aprobados`. Incluye datos semilla en `backend/db.sql`.
- **Autenticación:** JWT **Bearer** (expira en 1 hora). En cliente se guarda en `localStorage` (`token`, `registroAcademico`).

- **Comunicación:** REST JSON `http://localhost:4000/...`

2.1 Diagrama (alto nivel)

```
React (Pages & Components) ──HTTP/JSON──▶ Express API ──SQL──▶ MySQL
        ▲                                      │
        └── localStorage (JWT) ◀──────────────┘
```

3. Estructura del proyecto

```
PI_Grupo6_Informe4/
├── backend/
│   ├── server.js
│   ├── db.sql
│   ├── .env.example
├── App/
│   ├── public/
│   │   ├── index.html
│   │   ├── favicon.ico
│   │   ├── logo192.png
│   │   ├── logo512.png
│   │   ├── manifest.json
│   │   └── robots.txt
│   ├── src/
│   │   ├── pages/
│   │   │   ├── LoginPage.jsx
│   │   │   ├── RegisterPage.jsx
│   │   │   ├── ForgotPassword.jsx
│   │   │   ├── Home.jsx
│   │   │   ├── CrearPublicacion.jsx
│   │   │   └── Perfil.jsx
│   │   ├── App.js
│   │   ├── index.js
│   │   ├── home.css
│   │   ├── perfil.css
│   │   ├── publi.css
│   │   ├── index.css
│   │   └── tailwind.config.js
│   ├── package.json
│   ├── package-lock.json
```

```
│   ├── .env
│   └── postcss.config.js
├── package.json
├── package-lock.json
├── README.md
└── .gitignore
```

4. Backend (Express API)

4.1 Configuración principal

- **Entrypoint:** `backend/server.js`
- **Middlewares:** `cors()`, `bodyParser.json()`
- **Conexión MySQL:** `mysql2.createConnection(...)` (host, user, password, database).
  > *Sugerencia*: mover credenciales y `JWT_SECRET` a variables de entorno `.env`.
- **Puerto:** `4000` (mensaje: "Servidor corriendo en http://localhost:4000")
- **JWT:** firma con secreto literal `"variable_secreta"` y `expiresIn: "1h"`.

4.2 Esquema de base de datos (MySQL)

- **usuarios**
  - `id` (PK, AI), `registro_academico` (UNIQUE), `nombres`, `apellidos`, `correo` (UNIQUE), `password` (hash), `fecha_registro`.
- **cursos**
  - `id` (PK), `nombre`, `creditos`.

- **catedraticos**
  - `id` (PK), `nombre`.
- **publicaciones**
  - `id` (PK), `registro_academico` (FK → usuarios.registro_academico), **`curso_id` (nullable)**, **`catedratico_id` (nullable)**, `mensaje`, `fechaCreacion`.
  - **Check lógico:** debe existir **curso_id o catedratico_id**.
- **comentarios**
  - `id` (PK), `publicacion_id` (FK), `registro_academico` (FK), `mensaje`, `fechaCreacion`.
- **cursos_aprobados**
  - `id` (PK), `registro_academico` (FK), `curso_id` (FK), `creditos`.
  - **Índice único** (`registro_academico`, `curso_id`) para evitar duplicados.
- **Datos semilla:** lista extensa de cursos con créditos y catedráticos en `db.sql`.

4.3 Endpoints REST (resumen)

- **POST** `/api/registro`
- **POST** `/api/login`
- **POST** `/api/recuperar_password`
- **GET** `/api/publicaciones`
- **POST** `/api/publicaciones`
- **GET** `/api/publicaciones/:postId/comentarios`
- **POST** `/api/publicaciones/:postId/comentarios`
- **GET** `/api/usuarios/:registroAcademico`
- **GET** `/api/usuarios/:registroAcademico/cursos`
- **GET** `/api/cursos`
- **GET** `/api/catedraticos`
- **POST** `/api/usuarios/:registroAcademico/cursos-aprobados`

Autenticación requerida (Bearer JWT): creación de publicaciones, creación de comentarios, alta de cursos aprobados.
Públicos: login, registro, recuperar contraseña, listar publicaciones, catálogo de cursos y catedráticos, consultar perfil.

4.3.1 Autenticación

- **POST** `/api/registro`
  Body: `{ registroAcademico, nombres, apellidos, correo, password }`
  Lógica: valida campos, `bcrypt.hash(password, 10)`, inserta usuario. Maneja `ER_DUP_ENTRY`.
- **POST** `/api/login`
  Body: `{ registroAcademico, password }`

Lógica: busca usuario, compara con `bcrypt.compare`, emite JWT: `payload = { registroAcademico }`, secreto `"variable_secreta"`, expira en **1h**.
Respuesta típica: `{ token, nombres }`

- **POST** `/api/recuperar_password`
  Body: `{ registroAcademico, correo, nuevaPassword }`
  Lógica: verifica match de RA+correo, hace `hash(nuevaPassword)`, `UPDATE`.

4.3.2 Publicaciones y comentarios

- **GET** `/api/publicaciones`
  Respuesta: lista con `id, usuario (nombres), curso (opcional), catedratico (opcional), mensaje, fechaCreacion` (JOINs con usuarios/cursos/catedraticos). Orden descendente por fecha.
- **POST** `/api/publicaciones` *(JWT)*
  Body: `{ tipo: "curso"|"catedratico", valorId: number, mensaje: string }`
  Lógica: toma `registroAcademico` desde JWT; inserta en publicaciones usando `curso_id` *o* `catedratico_id` según `tipo`.
- **GET** `/api/publicaciones/:postId/comentarios`
  Respuesta: `{ id, mensaje, fechaCreacion, usuario }` (JOIN con usuarios).
- **POST** `/api/publicaciones/:postId/comentarios` *(JWT)*
  Body: `{ mensaje }`; inserta comentario vinculado al `postId` y al usuario del JWT.

4.3.3 Perfil y cursos

- **GET** `/api/usuarios/:registroAcademico` → datos básicos del usuario.

- **GET** `/api/usuarios/:registroAcademico/cursos` → cursos aprobados del usuario y total de créditos (el código calcula y devuelve `{ cursos, totalCreditos }`).

- **POST** `/api/usuarios/:registroAcademico/cursos-aprobados` *(JWT)* **Body:** `{ cursoId }`; toma créditos del curso y crea registro en `cursos_aprobados`.
  Conflictos: devuelve **409** si el curso ya estaba aprobado.

4.4 Consideraciones de seguridad

- **Hashing** de contraseñas con `bcrypt` (cost 10).

- **JWT** con vencimiento 1h. **Recomendado**: mover el secreto a `process.env.JWT_SECRET`.

- **Validación de entrada**: se validan campos requeridos y errores de DB; se sugiere fortalecer sanitización y limitar tamaño de campos.

- **CORS** habilitado para el origen del frontend.

5. Frontend (React)

5.1 Entradas principales

- **`src/index.js`**: Renderiza `<App />` con `React.StrictMode`.
- **`src/App.js`**: Define rutas con `react-router-dom 6`:
- `/` → `LoginPage`
- `/registro` → `RegisterPage`
- `/recuperar_password` → `ForgotPassword`
- `/home` → `Home`
- `/crear-publicacion` → `CrearPublicacion`
- `/perfil/:registroAcademico` → `Perfil`

5.2 Páginas y lógica clave

a) `LoginPage.jsx`

- **Estado**: `registroAcademico`, `password`, `loading`, `error`.
- **Acciones**: `handleLogin(e)` → `POST /api/login`.
  - On success: guarda **`token`** y **`registroAcademico`** en `localStorage` y navega a **`/home`**.
  - Manejo de errores de red/credenciales.

b) `RegisterPage.jsx`

- **Form** controlado: `{ registroAcademico, nombres, apellidos, correo, password }`.
- **Acción**: `POST /api/registro`. Muestra mensajes de éxito/error.

c) `ForgotPassword.jsx`

- **Form**: `{ registroAcademico, correo, nuevaPassword }` → `POST /api/recuperar_password`.

d) `Home.jsx`

- **Carga inicial**: `GET /api/publicaciones` con `Authorization: Bearer <token>`; ordena por `fechaCreacion` desc.
- **Filtros**: por **tipo** (`curso | catedratico`) + **valor**; además **búsqueda por usuario**.

- **Comentarios**:
  - `GET /api/publicaciones/:postId/comentarios` (toggle de visibilidad por publicación).
  - `POST /api/publicaciones/:postId/comentarios` (requiere JWT).
- **Sesión**: botón **Cerrar Sesión** (limpia `localStorage` y redirige).
- **Atajos de navegación**: ir a perfil del usuario logueado; ir a crear publicación.

e) `CrearPublicacion.jsx`

- **Catálogos**: `GET /api/cursos` y `GET /api/catedraticos`.
- **Form**: `tipo` (curso/catedratico) → muestra `select` dinámico (`valorId`) + `mensaje`.
- **Acción**: `POST /api/publicaciones` con JWT. Muestra feedback y permite **Volver a /home**.

f) `Perfil.jsx`

- **Route param**: `:registroAcademico`.
- **Consultas**:
  - `GET /api/usuarios/:registroAcademico` (datos del perfil).
  - `GET /api/usuarios/:registroAcademico/cursos` → `{ cursos, totalCreditos }`.
  - `GET /api/cursos` para llenar combo.
- **Agregar curso aprobado**: `POST /api/usuarios/:registroAcademico/cursos-aprobados` (JWT).
- **Cálculo**: muestra **total de créditos**. Restringe acciones a **mi propio perfil** (decodificando JWT con `jwt-decode`).

5.3 Estado, almacenamiento y navegación

- **Estado local** con `useState` y **efectos** con `useEffect`.
- **Sesión**: `localStorage["token"]`, `localStorage["registroAcademico"]`.
- **Navegación** con `useNavigate()`, `useParams()` y `<Routes />` (v6).

5.4 Estilos

- Archivos CSS dedicados: `home.css`, `perfil.css`, `publi.css`.
- Se observan clases tipo “utilitarias” (similar a Tailwind) definidas manualmente en CSS.

6. Instalación y ejecución

6.1 Requisitos

- **Node.js** v18+
- **MySQL** 8+ (o compatible)
- **NPM** (o Yarn/Pnpm)

6.2 Backend

1. Instalar dependencias en la **raíz** del proyecto:

   ```
   npm install
   ```

2. Importar esquema y datos:

   ```
   -- En MySQL
   SOURCE backend/db.sql;
   ```

3. Configurar (opcional recomendado) `.env` en la raíz:

   ```
   DB_HOST=localhost
   DB_USER=root
   DB_PASSWORD=
   DB_NAME=practica_web
   JWT_SECRET=variable_secreta
   PORT=4000
   ```

4. Ejecutar el servidor:

   ```
   node backend/server.js
   ```

   Si usas `import`/ESM, añade `"type": "module"` en `package.json` o usa Node v18+.

6.3 Frontend

1. Instalar dependencias:

   ```
   cd App npm
   install
   ```

2. Ejecutar CRA:

   `npm start`

   - Frontend: `http://localhost:3000`
   - Backend: `http://localhost:4000`

7. Pruebas y calidad

- CRA incluye `react-scripts test` y librerías de Testing Library. Aún no hay pruebas implementadas.

- Recomendado agregar pruebas de integración de API (Supertest) y pruebas E2E (Playwright/Cypress).

8. Solución de problemas (FAQ)

- **401 No autorizado**: Token ausente, inválido o expirado → iniciar sesión de nuevo.

- **ECONNREFUSED / MySQL**: verifica credenciales, que MySQL esté activo y que la base `practica_web` exista.

- **CORS**: si cambias origen/puerto del frontend, ajusta `cors()` en backend.

- **"address already in use :4000"**: el puerto está ocupado → cierra procesos o cambia `PORT`.

- **ESM/`import` no reconocido**: añade `"type": "module"` al `package.json` de la raíz o transpila.

- **DUPLICATE entry al aprobar curso**: la API retorna **409**; maneja feedback en UI.

9. Recomendaciones y mejoras futuras

- Variables de entorno para DB y JWT; rotación de secretos.

- Persistir sesión de forma más segura (httponly cookies + CSRF).

- Validación/sanitización adicional (cele/joi/zod) en todas las rutas.

- Paginación y caching en `/api/publicaciones`.

- Servicio de API en el frontend (una capa `api.js`) para centralizar `fetch` y manejo de errores.

- Manejo de roles (admin/estudiante) y eliminación/edición de publicaciones/comentarios.

- Tests automáticos (unitarios y E2E).

10. Anexos — Ejemplos de uso de API

Login

```
POST /api/login
Content-Type: application/json

{
  "registroAcademico": "1234567",
  "password": "••••••••"
}
```

Respuesta (200):

```
{ "token": "<JWT>", "nombres": "Juan" }
```

Crear publicación por curso

```
POST /api/publicaciones
Authorization: Bearer <JWT>
Content-Type: application/json

{ "tipo": "curso", "valorId": 8, "mensaje": "¿Alguien tiene los apuntes?"
}
```

Agregar comentario

```
POST /api/publicaciones/12/comentarios
Authorization: Bearer <JWT>
Content-Type: application/json

{ "mensaje": "Te los paso por correo." }
```

Consultar perfil

```
GET /api/usuarios/202300553
```

Agregar curso aprobado

```
POST /api/usuarios/202300553/cursos-aprobados
Authorization: Bearer <JWT>
Content-Type: application/json

{ "cursoId": 15 }
```

11. Dependencias

Backend (raíz `package.json`): { “bcrypt”: “^6.0.0”, “body-parser”: “^2.2.0”, “cors”: “^2.8.5”, “dotenv”: “^17.2.2”, “express”: “^5.1.0”, “jsonwebtoken”: “^9.0.2”, “mysql2”: “^3.14.5”, “react”: “^18.3.1”, “react-dom”: “^18.3.1”, “react-router-dom”: “^6.30.1” }

Frontend (`myapp/package.json`): { “@testing-library/dom”: “^10.4.1”, “@testing-library/jest-dom”: “^6.8.0”, “@testing-library/react”: “^16.3.0”, “@testing-library/user-event”: “^13.5.0”, “jwt-decode”: “^4.0.0”, “react”: “^18.3.1”, “react-dom”: “^18.3.1”, “react-router-dom”: “^6.30.1”, “react-scripts”: “^5.0.1”, “web-vitals”: “^2.1.4” }